UNIDAD 9. Acceso a bases de datos relacionales

Índice

1. Arquitectura de Acceso a Datos 2
   1.1. Interfaces de acceso a bases de datos: El estándar JDBC 2
   1.2. El ecosistema de Drivers 3
   1.3. La jerarquía del paquete `java.sql`: Interfaces clave 5
2. Gestión de Conexiones 8
   2.1. Establecimiento de conexiones: Uso de la clase `DriverManager` 8
   2.2. La URL de conexión (Connection String) 9
   2.3. Gestión de credenciales y seguridad: Configuración de usuario y contraseña 12
   2.4. Control de excepciones y cierre con `try-with-resources` 14
3. Ejecución de Consultas y Comandos 16
   3.1. Tipos de Sentencias: Diferencias entre `Statement` y `PreparedStatement` 16
   3.2. Seguridad y Rendimiento: Prevención de ataques y Optimización 18
   3.3. Ejecución de consultas (DQL) 20
   3.4. Ejecución de comandos de actualización (DML) 22
4. Recuperación y Procesamiento de Información 24
   4.1. El objeto `ResultSet`: Concepto de cursor y navegación 24
   4.2. Obtención de datos: Métodos `getXXX()` y Mapeo de Tipos 26
   4.3. Metadatos: Obtención de información sobre la estructura 28
5. Manipulación Avanzada de Información 30
   5.1. Transacciones en Java: `setAutoCommit`, `commit` y `rollback` 30
   5.2. Integridad de los datos: Asegurar la consistencia 33

Resumen 35
   1. Los Pilares de JDBC (Arquitectura) 35
   2. Estableciendo la Conexión 35
   3. Ejecución de Sentencias (La Regla de Oro) 35
   4. Métodos de Ejecución 35
   5. Recuperación de Datos (`ResultSet`) 35
   6. Integridad y Transacciones 36

1. Arquitectura de Acceso a Datos

1.1. Interfaces de acceso a bases de datos: El estándar JDBC

JDBC son las siglas de **Java Database Connectivity**. Es una API (Application Programming Interface) incluida en el JDK estándar que permite a los programas Java interactuar con cualquier base de datos relacional utilizando el lenguaje SQL.

El Problema: La Torre de Babel

Imagina que quieres escribir una aplicación.

- Si usas **MySQL**, el código para conectarse es de una forma.
- Si mañana cambias a **Oracle**, el código interno de Oracle es totalmente distinto.
- Si te pasas a **PostgreSQL**, otra vez cambia todo.

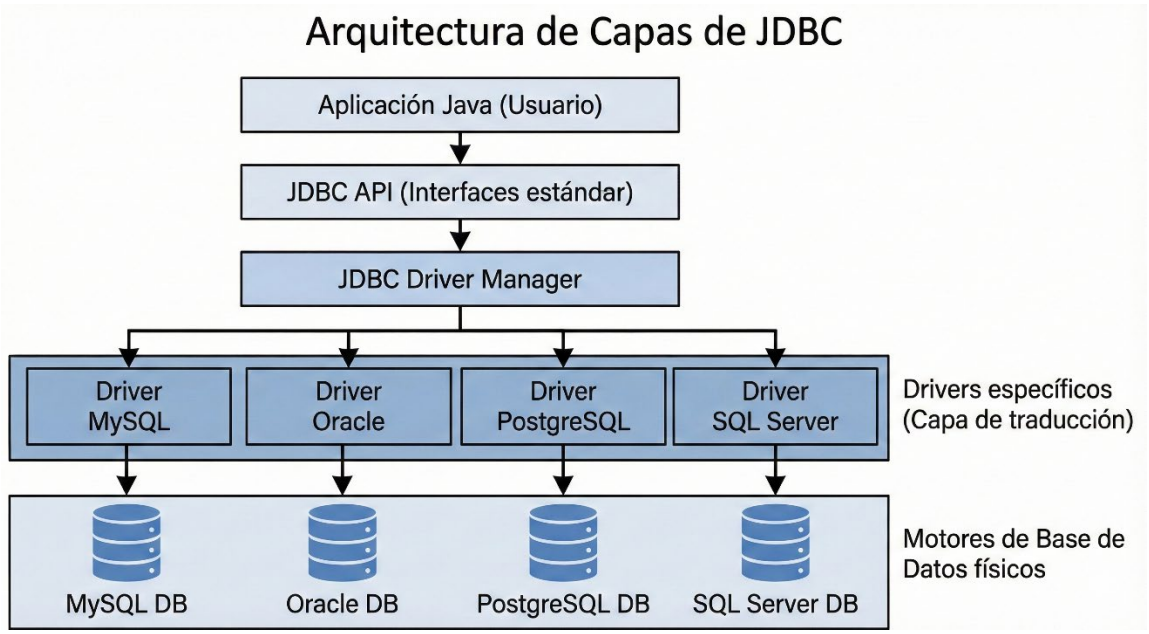
Sin JDBC, tendrías que reescribir todo tu programa cada vez que cambias de base de datos. Sería insostenible.

La Solución: Una Capa de Abstracción

JDBC actúa como un **traductor universal**. Tú escribes tu código Java utilizando las instrucciones estándar de JDBC (que son siempre las mismas), y JDBC se encarga de traducir esas instrucciones al "idioma nativo" de la base de datos específica que estés usando.

Esta arquitectura se basa en capas:

1. **Tu Aplicación Java:** Donde usas las clases del paquete `java.sql`.
2. **JDBC API:** La interfaz común (el contrato).
3. **JDBC Driver Manager:** El gestor que selecciona el traductor adecuado.
4. **JDBC Driver (Controlador):** El traductor específico (este es el software que cambia según la base de datos).
5. **La Base de Datos:** Donde viven los datos.



La Clave: Todo son Interfaces

Lo más importante que debes entender de este punto es que **JDBC es, en su mayoría, un conjunto de Interfaces**, no de clases concretas.

Java (Sun/Oracle) define las **reglas** (Interfaces):

- Define una interfaz llamada `Connection`: "Cualquier base de datos que quiera trabajar con Java debe tener una forma de conectarse que siga estas reglas".
- Define una interfaz llamada `Statement`: "Cualquier base de datos debe tener una forma de ejecutar SQL así".

Los **Fabricantes de Bases de Datos** (MySQL, Microsoft, etc.) escriben la **implementación** (Drivers):

- El equipo de MySQL escribe una clase que implementa `Connection` y hace que funcione con MySQL.
- El equipo de PostgreSQL escribe su propia clase que implementa `Connection` y hace que funcione con Postgres.

  Analogía del Enchufe: JDBC es como el diseño de un enchufe de pared.

  - **Java** dice: "El enchufe debe tener dos agujeros redondos".
  - **Tu código** es la tostadora: Solo sabe que necesita un enchufe de dos agujeros. No le importa qué hay detrás.
  - **La Base de Datos** es la central eléctrica: Puede ser nuclear, eólica o de carbón.
  - **El Driver** es el cableado que conecta tu enchufe estándar con esa central específica.

Ventajas de este estándar

1. **Independencia:** Puedes cambiar la base de datos de tu proyecto (de MySQL a Oracle) casi sin tocar tu código Java. Solo cambias el "Driver".
2. **Simplicidad:** Solo necesitas aprender un conjunto de comandos (`java.sql`) para dominar todas las bases de datos.
3. **Potencia:** Permite enviar sentencias SQL complejas directamente desde Java.

1.2. El ecosistema de Drivers

Como dijimos, JDBC es solo un conjunto de **interfaces** (contratos vacíos). Para que el código funcione, necesitamos una implementación real que haga el trabajo sucio. Esa implementación es el **Driver JDBC**.

1. ¿Qué es físicamente un Driver?

En el mundo Java, un Driver no es un programa instalable (.exe) ni una configuración del sistema operativo.

Es, sencillamente, un archivo `.jar` (Java ARchive).

Es una librería (biblioteca) externa que debes añadir al *Classpath* de tu proyecto (o a las dependencias de Maven/Gradle).

- Si usas MySQL, necesitas el archivo `mysql-connector-java-x.x.x.jar`.
- Si usas PostgreSQL, necesitas el archivo `postgresql-x.x.x.jar`.
- Si usas Oracle, necesitas `ojdbc.jar`.

2. Tipos de Controladores (La evolución)

Históricamente, han existido 4 tipos de drivers según cómo se conectaban a la base de datos. Hoy en día, casi siempre usarás el **Tipo 4**, pero es bueno conocer la historia para entender por qué es el mejor.

Tipo 1: Puente JDBC-ODBC (Obsoleto)

- **Funcionamiento:** Usaba el estándar antiguo de Microsoft (ODBC) para conectarse.
- **Problema:** Lento y dependiente del Sistema Operativo (solo funcionaba bien en Windows). Desapareció en Java 8.

Tipo 2: API Nativa (Native API)

- **Funcionamiento:** El driver Java llama a librerías escritas en C/C++ instaladas en el ordenador cliente.
- **Problema:** Necesitas instalar software específico de la base de datos en cada ordenador que use tu programa. Si falla la librería en C, se cuelga toda la JVM.

Tipo 3: Protocolo de Red (Middleware)

- **Funcionamiento:** Java se conecta a un "Servidor Intermedio", y este se conecta a la base de datos.
- **Uso:** Muy complejo. Solo se usa en arquitecturas empresariales muy específicas.

Tipo 4: Protocolo Nativo Puro (Pure Java) - EL ESTÁNDAR ACTUAL

- **Funcionamiento:** Es el más usado hoy en día. El driver está escrito 100% en Java.
- **La Magia:** El driver conoce el protocolo de red binario de la base de datos (habla directamente con el puerto 3306 de MySQL o 5432 de Postgres) usando Sockets.
- **Ventaja:** No necesitas instalar nada en el ordenador del cliente. Solo copias el `.jar` junto con tu aplicación y funciona en Windows, Linux, Mac o Android.

3. ¿Cómo actúan de "Traductores"?

El proceso interno cuando ejecutas una consulta es el siguiente:

1. **Tu Código (Java):** Llama a un método estándar, por ejemplo: `conexion.createStatement()`.
2. **La Interfaz JDBC:** Pasa la orden al Driver cargado.
3. **El Driver (El Traductor):**

- Coge esa orden Java.
- La convierte en paquetes de bytes que la base de datos entienda (protocolo propietario).
- Envía los bytes por la red (TCP/IP).

4. **La Base de Datos:** Recibe los bytes, ejecuta la SQL y devuelve los resultados en su formato binario.
5. **El Driver:**

   - Recibe la respuesta binaria.
   - La convierte en objetos Java (`ResultSet`, `String`, `Integer`).

6. **Tu Código:** Recibe los objetos Java limpios y listos para usar.

Drivers Comunes

Para conectar Java, necesitarás saber el nombre de la clase principal del driver (aunque en versiones modernas de Java se detecta sola) y tener el `.jar`.

Base de Datos	Nombre del JAR típico	Clase del Driver (Histórico)
MySQL	`mysql-connector-j-8.x.jar`	`com.mysql.cj.jdbc.Driver`
PostgreSQL	`postgresql-42.x.jar`	`org.postgresql.Driver`
Oracle	`ojdbc8.jar`	`oracle.jdbc.OracleDriver`
SQL Server	`mssql-jdbc.jar`	`com.microsoft.sqlserver.jdbc.SQLServerDriver`
H2 / SQLite	`h2.jar` / `sqlite-jdbc.jar`	`org.h2.Driver`

1.3. La jerarquía del paquete `java.sql`: Interfaces clave

El paquete `java.sql` contiene docenas de clases, pero el 90% del tiempo trabajarás con tres protagonistas principales que funcionan en cadena. No puedes tener uno sin el anterior.

Para entenderlo, visualiza el flujo de datos como una carretera:

1. `Connection` (La Carretera)

Representa una **sesión física activa** con la base de datos.

- **Función:** Es el túnel que conecta tu aplicación Java con el servidor MySQL/Oracle. Sin esto, no hay comunicación.
- **Responsabilidades:**
  - Crear los objetos `Statement`.
  - Gestionar **Transacciones**: Aquí es donde decides si haces `commit()` (guardar cambios) o `rollback()` (deshacer todo).
- **Ciclo de vida:** Abrir una conexión es una operación "cara" (tarda tiempo y consume recursos). Por eso, se abren, se usan y se cierran lo antes posible.

Analogía: Es como **descolgar el teléfono** y llamar al restaurante. Mientras la línea está abierta (`Connection`), puedes hacer pedidos. Si cuelgas (`close`), se acabó la sesión.

2. `Statement` (El Vehículo)

Es el objeto encargado de **transportar tu sentencia SQL** a través de la conexión hacia la base de datos.

- **Función:** Envía comandos SQL (SELECT, INSERT, UPDATE, DELETE) y recibe los resultados.
- **Tipos principales:**

  1. **`Statement`:** El básico. Envía SQL texto plano. (Poco seguro, vulnerable a inyección SQL).
  2. **`PreparedStatement`: (El Recomendado)**. Precompila la consulta y usa parámetros (`?`). Es más rápido y seguro.
  3. **`CallableStatement`:** Para llamar a Procedimientos Almacenados (funciones guardadas dentro de la base de datos).

  Analogía: Es el **camión de reparto**. Tú cargas el camión con tu pedido ("Quiero una pizza de pepperoni" - *SQL*) y el camión viaja por la carretera (`Connection`) hasta el restaurante.

3. `ResultSet` (La Carga)

Es el objeto que contiene **los datos que la base de datos devuelve** tras una consulta de lectura (`SELECT`).

- **Estructura:** Imagínalo como una hoja de cálculo virtual en la memoria de Java, con filas y columnas.
- **El Cursor:** El `ResultSet` no te da todos los datos de golpe. Mantiene un puntero (cursor) que inicialmente está *antes* de la primera fila.
- **Navegación:** Usas el método `next()` para mover el cursor a la siguiente fila. Si devuelve `true`, hay datos; si devuelve `false`, has terminado.

  Analogía: Es la **caja de la pizza** que trae el camión de vuelta. Abres la caja y vas cogiendo las porciones una a una (`next()`).

El Flujo de Dependencia (La Regla de Oro)

En código, verás que estos objetos se crean **uno a partir del otro**, como muñecas rusas o una cadena de montaje:

Java

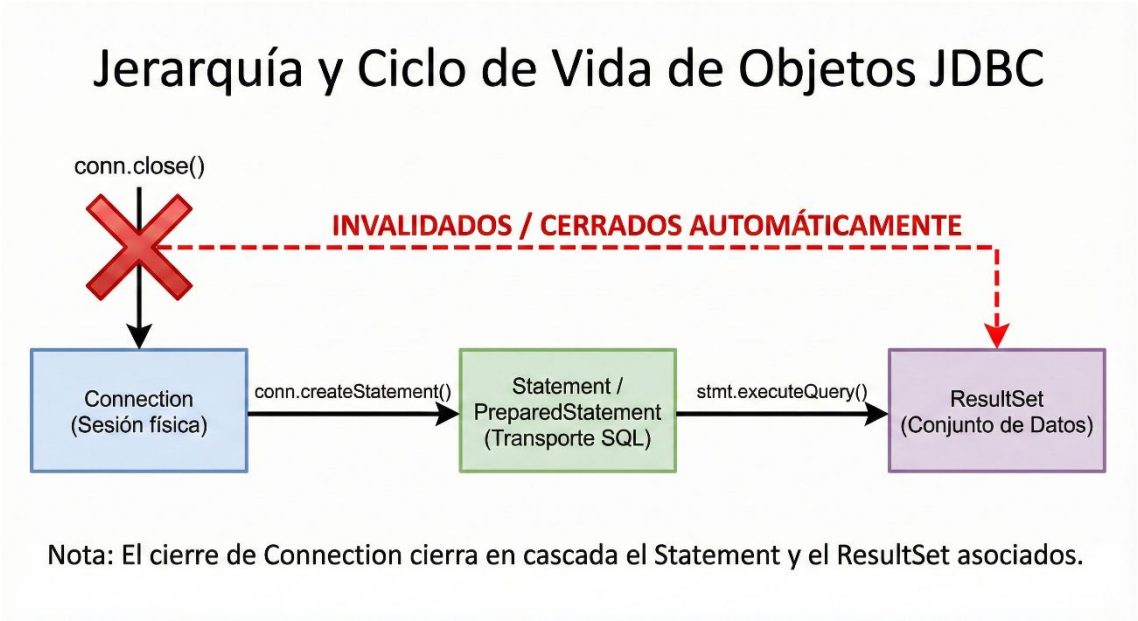
```
// 1. Connection: La fábrica de Statements
Connection conn = DriverManager.getConnection(url, user, pass);

// 2. Statement: Creado por la Connection
Statement stmt = conn.createStatement();

// 3. ResultSet: Generado por la ejecución del Statement
ResultSet rs = stmt.executeQuery("SELECT * FROM clientes");

// ... procesar ...
```

Importante: Si cierras la `Connection`, automáticamente mueren el `Statement` y el `ResultSet` asociados. Son dependientes.



Resumen de la Jerarquía

Interfaz	Rol	Método clave para obtenerlo
`Connection`	Sesión / Transacción	`DriverManager.getConnection(...)`
`Statement`	Ejecutor de SQL	`conn.createStatement()`
`ResultSet`	Contenedor de Datos	`stmt.executeQuery("SQL")`

2. Gestión de Conexiones

2.1. Establecimiento de conexiones: Uso de la clase `DriverManager`

La clase **`java.sql.DriverManager`** actúa como el recepcionista jefe. Es una clase estática que gestiona todos los drivers JDBC que tienes disponibles en tu proyecto.

Su función principal es sencilla: tú le pides una conexión (dándole la dirección, usuario y contraseña), y él busca entre todos los drivers registrados cuál es capaz de entender esa dirección y conectar.

El Método Estrella: `getConnection()`. Para obtener un objeto `Connection`, invocamos al método estático `DriverManager.getConnection`. Existen tres variantes principales, pero la más común es esta:

Java
```
Connection conn = DriverManager.getConnection(url, usuario,
password);
```

Si todo va bien, te devuelve un objeto Connection listo para usar. Si algo falla (la base de datos está caída, la contraseña es incorrecta, o no tienes el driver), lanza una SQLException.

Pasos para conectar (El patrón clásico)

Aunque en las versiones modernas de Java (JDBC 4.0 en adelante) algunos pasos se han automatizado, es importante conocer el proceso completo:

1. **Cargar el Driver (Opcional en Java moderno):** Antiguamente, tenías que decirle a Java explícitamente "¡Carga la clase del driver de MySQL!". Esto se hacía con Class.forName(). Hoy en día, Java suele ser capaz de encontrar el driver automáticamente si el .jar está en el classpath, pero verás este código en muchos tutoriales antiguos.

Java
```
// Código "Legado" (ya no es estrictamente necesario en Java 8+)
Class.forName("com.mysql.cj.jdbc.Driver");
```

2. **Pedir la conexión:** Llamamos al DriverManager pasando los datos de conexión.

Ejemplo de Código: Conectando a una Base de Datos

Vamos a ver un ejemplo real intentando conectar a una base de datos MySQL local llamada "tienda".

Java
```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class TestConexion {
    public static void main(String[] args) {

        // 1. Definimos los datos de conexión
```

```
        // (Veremos qué es este String raro "jdbc:mysql..." en el
siguiente punto)
        String url = "jdbc:mysql://localhost:3306/tienda";
        String usuario = "root";
        String password = "admin";

        Connection conexion = null;

        try {
            // 2. Intentamos conectar
            System.out.println("Conectando a la base de datos...");

            conexion = DriverManager.getConnection(url, usuario,
password);

            // Si llegamos aquí, ¡éxito!
            System.out.println("¡Conexión establecida
correctamente!");

            // Aquí iría tu lógica (SELECT, INSERT...)

        } catch (SQLException e) {
            // 3. Gestión de errores
            System.err.println("Error al conectar: " +
e.getMessage());
            // Es útil imprimir el código de error SQL específico
(ej: 1045 para acceso denegado)
            System.err.println("Código SQLState: " +
e.getSQLState());

        } finally {
            // 4. SIEMPRE cerrar la conexión (si se abrió)
            try {
                if (conexion != null) {
                    conexion.close();
                    System.out.println("Conexión cerrada.");
                }
            } catch (SQLException e) {
                System.err.println("Error al cerrar: " +
e.getMessage());
            }
        }
    }
}
```

Más info: Este bloque `finally` para cerrar es la forma antigua. En el punto 9.2.4 veremos cómo hacerlo mucho más limpio con `try-with-resources`.

2.2. La URL de conexión (Connection String)

Para que el `DriverManager` sepa qué driver activar y a qué servidor llamar, necesitamos pasarle una cadena de texto con un formato muy específico. A esto se le llama **JDBC URL**.

Aunque se parece a una dirección web (`http://...`), tiene su propia sintaxis.

1. Anatomía de una URL JDBC

Todas las URLs de conexión siguen, en general, esta estructura tripartita:

1. **Protocolo (`jdbc:`):** Siempre es igual. Le indica a Java que estamos hablando de bases de datos.
2. **Sub-protocolo (`mysql:`, `postgresql:`, `oracle:`):** Aquí es donde ocurre la magia. Esta palabra clave le dice al `DriverManager` qué archivo `.jar` (qué driver) debe encargarse de la petición.

    - Si pones `jdbc:mysql:...`, Java buscará el driver de MySQL. Si no tienes el `.jar` de MySQL en tu proyecto, fallará.

3. **Sub-nombre (Identificador de la fuente):** Es la dirección física. Generalmente sigue el formato `//servidor:puerto/nombre_base_datos`, aunque varía según el fabricante.

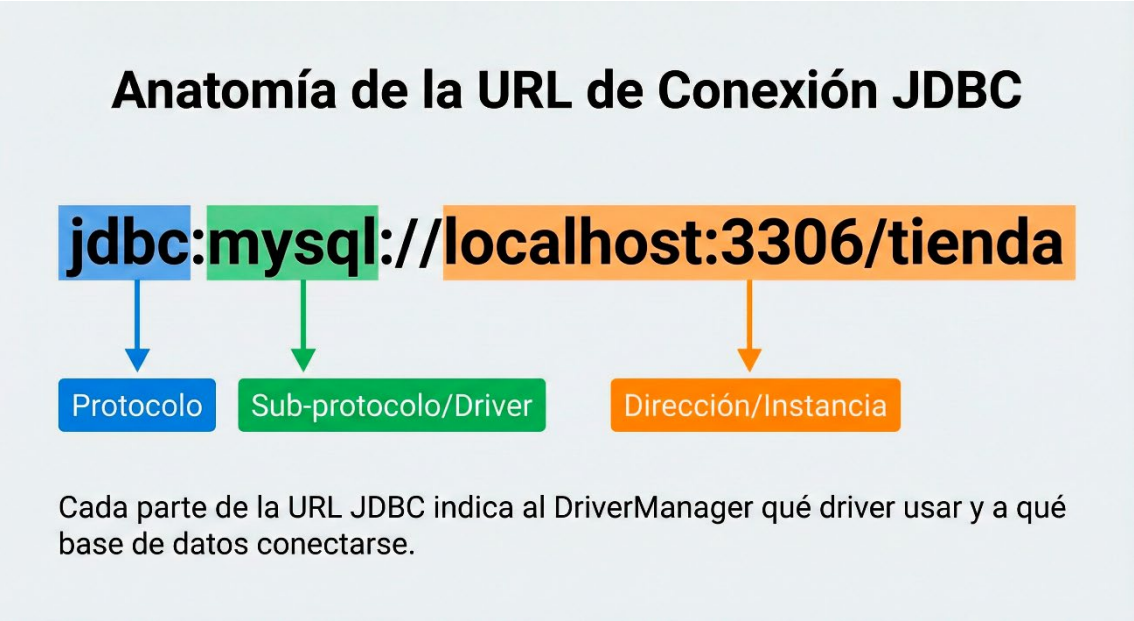
2. Ejemplos según el Motor de Base de Datos

Cada base de datos tiene sus peculiaridades (puertos por defecto, separadores, etc.). Aquí tienes la "chuleta" de las más comunes:

A. MySQL / MariaDB (El más común en educación)

- **Formato:** `jdbc:mysql://[host]:[puerto]/[base_datos]`
- **Puerto por defecto:** 3306
- **Ejemplo:**

Java
```
String url = "jdbc:mysql://localhost:3306/tienda_online";
```



B. PostgreSQL

- **Formato:** `jdbc:postgresql://[host]:[puerto]/[base_datos]`
- **Puerto por defecto:** 5432
- **Ejemplo:**

Java
```
String url = "jdbc:postgresql://192.168.1.50:5432/facturacion";
```

C. Oracle Database

Oracle es especial ("especialito"). Tiene varios tipos de drivers, pero el más usado es el **Thin Driver**. Además, no usa el concepto de "Base de datos" igual, sino **SID** o **Service Name**.

- **Formato (usando SID - antiguo):** `jdbc:oracle:thin:@[host]:[puerto]:[SID]`
- **Formato (usando Service Name - moderno):** `jdbc:oracle:thin:@[host]:[puerto]/[service_name]`
- **Puerto por defecto:** 1521
- **Ejemplo (Oracle XE):**

Java
```
// Nota los dos puntos ':' antes del xe
String url = "jdbc:oracle:thin:@localhost:1521:xe";
```

D. Microsoft SQL Server

- **Formato:** `jdbc:sqlserver://[host]:[puerto];databaseName=[nombre]`
- **Puerto por defecto:** 1433
- **Nota:** A diferencia de los demás que usan `/`, SQL Server suele usar `;` para separar propiedades.
- **Ejemplo:**

Java
```
String url =
"jdbc:sqlserver://localhost:1433;databaseName=empleados";
```

E. H2 / SQLite (Bases de datos en fichero)

Como no son servidores, no tienen "host" ni "puerto". La ruta es la ubicación del archivo en tu disco duro.

- **Ejemplo H2:** `jdbc:h2:~/test` (Carpeta de usuario) o `jdbc:h2:./datos/mi_db` (Ruta relativa).

3. Parámetros Adicionales (La configuración fina)

A menudo, la conexión básica no es suficiente. Necesitamos especificar la zona horaria, la codificación de caracteres o si usamos SSL.

Estos parámetros se añaden al final de la URL, siguiendo la sintaxis web: `?clave=valor&otraClave=otroValor`.

Caso Clásico: MySQL y la Zona Horaria. En versiones modernas del driver de MySQL, es obligatorio especificar la zona horaria o lanzará una excepción.

Java
```
String url = "jdbc:mysql://localhost:3306/tienda"
           + "?useUnicode=true"
           + "&useJDBCCompliantTimezoneShift=true"
           + "&useLegacyDatetimeCode=false"
           + "&serverTimezone=UTC"  // ¡Muy importante!
           + "&useSSL=false";       // Para desarrollo local,
desactivar SSL
```

Importante: Nunca intentes memorizar estas URLs largas de memoria. Ten a mano siempre un "snippet" o chuleta, o búscalas en la documentación oficial del driver cuando las necesites. Lo importante es entender qué significa cada parte para poder cambiar el nombre de la base de datos o el puerto cuando sea necesario.

2.3. Gestión de credenciales y seguridad: Configuración de usuario y contraseña

Una base de datos suele contener la información más sensible de una empresa. Si pones el usuario y la contraseña incorrectamente en tu código, estás dejando las llaves de la caja fuerte pegadas en la puerta.

Vamos a ver las tres formas de gestionar esto, de la menos segura a la más profesional.

1. El método básico (Directo en el método)

Es lo que hemos visto hasta ahora. Pasamos el usuario y la contraseña como argumentos `String` directamente al `DriverManager`.

Java
```
// Forma estándar para pruebas rápidas
String user = "root";
String pass = "1234";
Connection conn = DriverManager.getConnection(url, user, pass);
```

- **Ventaja:** Rápido y fácil de leer.
- **Desventaja:** Limitado. Si necesitas pasar más parámetros (como timeouts, codificación, SSL), la llamada al método se vuelve kilométrica.

2. El método estructurado (Objeto `Properties`)

La clase `java.util.Properties` es un diccionario de tipo Clave-Valor diseñado específicamente para configuraciones. JDBC acepta un objeto `Properties` en lugar de usuario y contraseña sueltos.

Esto es mucho más limpio porque puedes meter ahí toda la configuración extra.

Java
```
import java.util.Properties;
```

```
String url = "jdbc:mysql://localhost:3306/tienda";
Properties props = new Properties();

// Configuramos las credenciales dentro del objeto
props.put("user", "root");
props.put("password", "1234");

// Configuraciones extra (muy útil)
props.put("connectTimeout", "2000"); // Esperar máx 2 segundos
props.put("ssl", "false");

// Pasamos el objeto completo
Connection conn = DriverManager.getConnection(url, props);
```

3. El método Profesional (Externalización)

Aquí está la regla de oro de la seguridad en desarrollo: **NUNCA escribas contraseñas directamente en el código fuente (Hardcoding).**

¿Por qué es peligroso el Hardcoding?

1. **Ingeniería Inversa:** Los archivos `.class` (compilados) se pueden descompilar fácilmente. Si alguien obtiene tu `.jar`, podrá ver la contraseña en texto plano.
2. **Control de Versiones (Git):** Si subes tu código a GitHub con la contraseña escrita, quedará en el historial para siempre. Los bots escanean GitHub buscando contraseñas de bases de datos para secuestrarlas.

La Solución: Archivos de Propiedades Externos. Creamos un archivo de texto aparte (ej: `database.properties`) y hacemos que Java lo lea.

Archivo `database.properties` (en tu disco duro):
Properties
```
db.url=jdbc:mysql://localhost:3306/tienda
db.user=admin_ventas
db.password=Kj8#mA29!
```

Código Java (Genérico y Seguro):
Java
```
Properties config = new Properties();
// Cargamos el fichero externo
try (FileInputStream fis = new
FileInputStream("database.properties")) {
    config.load(fis);
}

// Leemos las claves del fichero
String url = config.getProperty("db.url");
// Las credenciales ya están dentro de 'config', así que lo pasamos
entero
Connection conn = DriverManager.getConnection(url, config);
```

Ventaja: Si cambias la contraseña de la base de datos, solo editas el archivo de texto. No tienes que volver a compilar todo el programa. Además, ese archivo se añade al `.gitignore` para no subirlo a la nube.
Consejo de Seguridad: El Principio del Mínimo Privilegio

Nunca uses el usuario `root` o administrador para conectar tu aplicación Java.

- Si un hacker rompe tu aplicación y estás conectado como `root`, podrá borrar todas las bases de datos (`DROP DATABASE`).
- **Lo correcto:** Crea un usuario específico para la aplicación (ej: `usuario_app`) que solo tenga permisos `SELECT`, `INSERT`, `UPDATE` y `DELETE` sobre las tablas que necesite, y nada más.

2.4. Control de excepciones y cierre con `try-with-resources`

Trabajar con bases de datos es, por definición, **inseguro**. Un cable de red puede desconectarse, el servidor puede estar saturado, la contraseña puede caducar o la consulta SQL puede tener un error de sintaxis.

Por eso, Java nos obliga a tratar con una excepción muy específica: **`SQLException`**.

1. La clase `SQLException`

Es una **Excepción Verificada (Checked Exception)**. Esto significa que el compilador **no te dejará compilar el código** si no envuelves tu lógica de base de datos en un bloque `try-catch` o declaras que tu método lanza la excepción (`throws SQLException`).

Una `SQLException` no es un simple mensaje de error; es un objeto complejo que trae mucha información útil para depurar:

- **`getMessage()`:** Descripción del error legible por humanos (ej: "Access denied for user 'root'").
- **`getSQLState()`:** Un código estándar de 5 caracteres (definido por el estándar X/OPEN) que es común entre bases de datos. Sirve para identificar el tipo de error (ej: "08001" suele ser error de conexión).
- **`getErrorCode()`:** El código de error específico del fabricante.
    - Ejemplo: En MySQL, el código `1045` es error de contraseña. En Oracle, sería `ORA-01017`.

2. El problema del cierre de recursos (La pesadilla del `finally`)

Como vimos antes, los objetos JDBC (`Connection`, `Statement`, `ResultSet`) mantienen abiertos canales de comunicación con el servidor. Si no los cierras explícitamente, provocas **fugas de memoria** (Memory Leaks) que pueden tumbar el servidor de base de datos al agotar las conexiones disponibles.

La Regla del Orden Inverso: Debes cerrar los recursos en el orden inverso al que los abriste:

1. Primero el `ResultSet` (el resultado).
2. Luego el `Statement` (la orden).
3. Finalmente, la `Connection` (la línea telefónica).

El Código Antiguo (Infierno de anidamiento): Antes de Java 7, hacer esto bien era horrible. Tenías que poner un `try-catch` dentro del `finally` para cada cierre, porque el propio método `close()` también puede lanzar una excepción.

3. La Solución Moderna: `try-with-resources`

Desde Java 7, cualquier clase que implemente la interfaz **`AutoCloseable`** (y todas las de JDBC lo hacen) puede gestionarse automáticamente.

La sintaxis consiste en declarar e inicializar los recursos **dentro de los paréntesis** del `try`.

- **Ventaja 1:** El código es mucho más limpio (te ahorras 15 líneas de `finally`).
- **Ventaja 2:** Java garantiza que se cerrarán **SIEMPRE**, haya éxito o haya error.
- **Ventaja 3:** Java los cierra automáticamente en el **orden correcto** (inverso a su declaración).

Ejemplo Perfecto de Conexión y Cierre

Este es el patrón que deberías usar en el 99% de tus desarrollos profesionales:

Java
```
import java.sql.*;

public class AccesoRobusto {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/tienda";
        String user = "vendedor";
        String pass = "secreto";
        String sql = "SELECT * FROM productos";

        // EL BLOQUE MÁGICO
        // Declaramos Connection, Statement y ResultSet dentro del
paréntesis.
        // Separamos cada recurso con punto y coma (;).
        try (
            Connection conn = DriverManager.getConnection(url,
user, pass);
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql)
        ) {
```

```
            // Procesamos los resultados (lo veremos en detalle en
9.4)
            while (rs.next()) {
                System.out.println("Producto: " +
rs.getString("nombre"));
            }

            // ¡MAGIA! Aquí no hace falta llamar a close().
            // Al llegar a esta llave '}', Java cierra rs, luego
stmt, luego conn.

        } catch (SQLException e) {
            // Manejo profesional del error
            System.err.println("--- ERROR DE BASE DE DATOS ---");
            System.err.println("Mensaje: " + e.getMessage());
            System.err.println("Estado SQL: " + e.getSQLState());
            System.err.println("Código Error Vendor: " +
e.getErrorCode());

        } catch (Exception e) {
            // Captura genérica por si falla algo que no sea SQL
            System.err.println("Error general: " + e.getMessage());
        }
    }
}
```

Nota: Fíjate que incluso el `ResultSet` (rs) se declara dentro del paréntesis del `try`. Esto asegura que se cierre antes que el `Statement`.

3. Ejecución de Consultas y Comandos

En JDBC, para ejecutar cualquier SQL, necesitamos un objeto que transporte esa orden. Java nos ofrece dos formas principales de hacerlo: la forma "básica" (`Statement`) y la forma "inteligente" (`PreparedStatement`).

3.1. Tipos de Sentencias: Diferencias entre `Statement` y `PreparedStatement`

Ambas interfaces sirven para lo mismo: enviar una sentencia SQL al servidor de base de datos. Sin embargo, la forma en que manejan los datos variables (parámetros) es radicalmente distinta.

1. `Statement` (La forma estática)

Es la interfaz más simple. Se utiliza principalmente para sentencias SQL **fijas**, que no llevan parámetros o variables.

- **Funcionamiento:** Envías la cadena SQL completa y finalizada.
- **Manejo de variables:** Si necesitas insertar una variable (ej: buscar un usuario por nombre), tienes que usar **Concatenación de Strings**.
- **Uso ideal:** Sentencias DDL (`CREATE TABLE`, `DROP TABLE`) o consultas estáticas (`SELECT * FROM configuracion`).

El problema de la concatenación: Mira este código. Es difícil de leer y propenso a errores (olvidar una comilla simple ' es muy fácil):

Java
```
// CÓDIGO NO RECOMENDADO (Statement)
String nombre = "Juan";
int edad = 25;

// Tienes que poner las comillas simples ' manualmente para el
texto
String sql = "INSERT INTO usuarios (nombre, edad) VALUES ('" +
nombre + "', " + edad + ")";

Statement stmt = conn.createStatement();
stmt.executeUpdate(sql);
```

2. `PreparedStatement` (La forma precompilada)

Es una sub-interfaz de `Statement`. Es la **opción recomendada** para el 99% de las consultas, especialmente si llevan variables.

- **Funcionamiento:** Envías una **plantilla** de SQL a la base de datos. En lugar de los valores reales, pones signos de interrogación `?` (comodines o *placeholders*).
- **Manejo de variables:** Usas métodos `set` (`setInt`, `setString`) para "rellenar" los huecos.
- **La Magia (Precompilación):** La base de datos recibe la plantilla, la analiza, crea un plan de ejecución y lo guarda en caché. Luego tú solo envías los datos. Esto hace que sea **más rápido** si ejecutas la misma sentencia muchas veces con datos distintos.

El código limpio:
Java
```
// CÓDIGO PROFESIONAL (PreparedStatement)
String sql = "INSERT INTO usuarios (nombre, edad) VALUES (?, ?)";

// 1. Preparamos la plantilla
PreparedStatement pstmt = conn.prepareStatement(sql);

// 2. Rellenamos los huecos (El índice empieza en 1, no en 0)
pstmt.setString(1, "Juan"); // Primer ?
pstmt.setInt(2, 25);        // Segundo ?

// 3. Ejecutamos (Fíjate que executeUpdate() aquí va sin
parámetros)
pstmt.executeUpdate();
```

Comparativa: ¿Por qué `PreparedStatement` gana por goleada?

Hay tres razones fundamentales por las que deberías olvidar `Statement` para consultas con datos:

A. Seguridad (Inyección SQL)

Esta es la razón más crítica.

- **`Statement`:** Si concatenas strings, un usuario malintencionado puede escribir código SQL dentro del campo de texto (ej: nombre = `' OR '1'='1`) y borrar tu base de datos o robar datos.
- **`PreparedStatement`:** Trata los datos estrictamente como datos, nunca como código ejecutable. Neutraliza automáticamente las comillas y caracteres peligrosos. **Es la vacuna principal contra la Inyección SQL.**

B. Rendimiento

- **`Statement`:** Cada vez que lo ejecutas, la base de datos tiene que leer la Query, comprobar la sintaxis y compilarla desde cero.
- **`PreparedStatement`:** La base de datos compila la estructura SQL **una sola vez**. Si tienes un bucle para insertar 1000 usuarios, la base de datos reutiliza el plan 1000 veces, ahorrando mucho tiempo de CPU.

C. Legibilidad

- **`Statement`:** El "infierno de las comillas" (`"'" + var + "'"`). Es muy difícil mantener el código.
- **`PreparedStatement`:** El SQL queda limpio y separado de los datos Java.

Tabla Resumen

Característica	Statement	PreparedStatement
Sintaxis SQL	Se construye concatenando Strings (`+`).	Se usa una plantilla con `?`.
Seguridad	**Vulnerable** a Inyección SQL.	**Seguro** (escapa datos automáticamente).
Rendimiento	Lento en repeticiones (recompila siempre).	Rápido (precompila y cachea).
Legibilidad	Mala (difícil ver errores de comillas).	Buena (SQL limpio).
Uso	Solo para DDL o consultas sin variables.	**Siempre** que haya variables.

3.2. Seguridad y Rendimiento: Prevención de ataques y Optimización

Como vimos, la elección entre `Statement` y `PreparedStatement` no es solo una cuestión de estilo. Afecta a la integridad de tus datos (Seguridad) y a la velocidad de tu aplicación (Rendimiento).

1. Seguridad: La pesadilla de la Inyección SQL

La **Inyección SQL** ocurre cuando un atacante logra manipular la estructura de tu consulta insertando código SQL malicioso a través de los campos de entrada de tu aplicación (formularios, URLs, etc.).

El escenario del crimen (Usando `Statement`)

Imagina un login muy simple donde concatenas cadenas.

Java
```
// CÓDIGO VULNERABLE (NO COPIAR)
String usuario = request.getParameter("user"); // Input del usuario
String sql = "SELECT * FROM usuarios WHERE nombre = '" + usuario +
"'";
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(sql);
```

El Ataque: Un usuario normal escribe Juan. La query es: `SELECT * FROM usuarios WHERE nombre = 'Juan'` (Todo bien).

Un atacante escribe: `Juan' OR '1'='1`

La query resultante que ejecuta la base de datos es: `SELECT * FROM usuarios WHERE nombre = 'Juan' OR '1'='1'`

El Resultado: Como `'1'='1'` es **siempre VERDADERO**, la base de datos devuelve todos los usuarios de la tabla. El atacante ha entrado sin saber la contraseña. Peor aún, podrían escribir `'; DROP TABLE usuarios; --` y borrar tus datos.

La Solución (Usando `PreparedStatement`)

Cuando usas `?`, separas la lógica de los datos.

Java
```
// CÓDIGO SEGURO
String sql = "SELECT * FROM usuarios WHERE nombre = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);

// El atacante escribe: Juan' OR '1'='1
pstmt.setString(1, "Juan' OR '1'='1");
```

El Resultado: La base de datos busca literalmente un usuario cuyo nombre sea exactamente la cadena de texto "`Juan' OR '1'='1`". Obviamente no existe, y no devuelve nada. La estructura SQL (`SELECT... WHERE...`) es inmutable; nada de lo que escriba el usuario puede cambiarla.

2. Rendimiento: La caché de planes de ejecución

Una base de datos es un software muy complejo. Cuando le envías una consulta SQL, no la ejecuta inmediatamente. Debe seguir un proceso costoso:

1. **Parsing (Análisis):** Comprueba que la sintaxis es correcta (¿Has puesto bien las comas? ¿Existe la tabla?).
2. **Compilation (Compilación):** Traduce el SQL a código máquina interno.

3. **Optimization (Planificación):** Decide cuál es la forma más rápida de buscar los datos (¿Uso un índice? ¿Escaneo toda la tabla?). Esto consume mucha CPU.
4. **Execution (Ejecución):** Busca los datos.

El problema con `Statement`

Si usas concatenación, cada query es diferente para la base de datos:

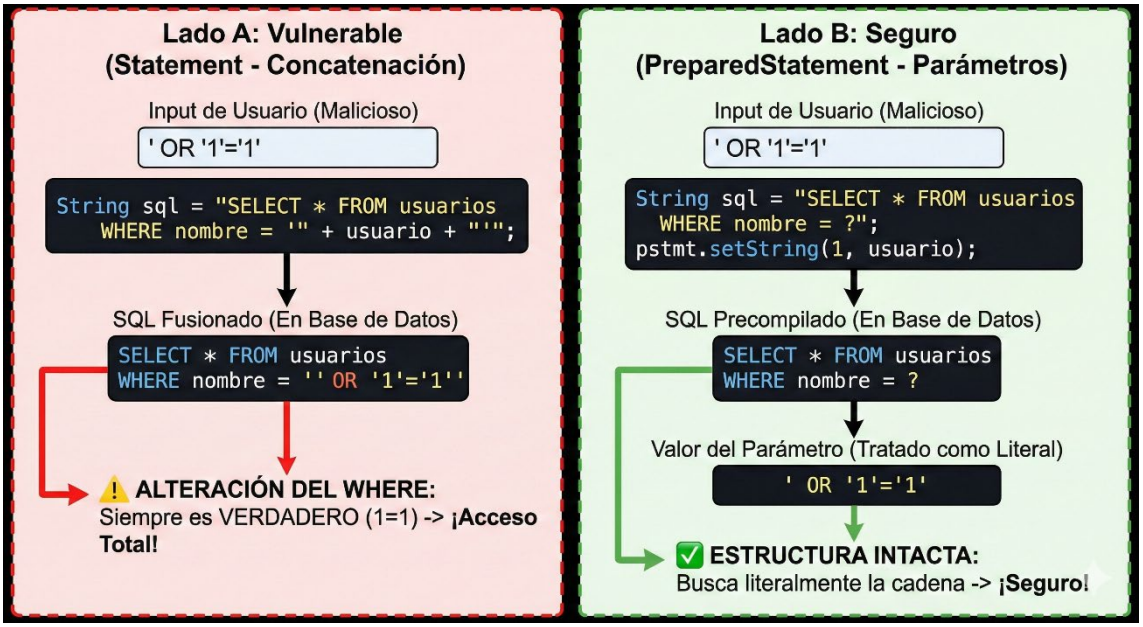
- Query 1: `SELECT * FROM producto WHERE id = 1`
- Query 2: `SELECT * FROM producto WHERE id = 2`

Para la base de datos, son dos frases distintas. Tiene que hacer los pasos 1, 2 y 3 **cada vez**.

La ventaja de PreparedStatement

Envías la plantilla primero: `SELECT * FROM producto WHERE id = ?`

1. La base de datos hace el Parsing, Compilación y Optimización **UNA SOLA VEZ**.
2. Guarda ese "Plan de Ejecución" en su memoria caché.
3. Cuando luego envías el dato `1`, `2` o `500`, la base de datos salta directamente al paso 4 (Ejecución) reutilizando el plan ya calculado.
   Conclusión: En bucles grandes o aplicaciones con muchos usuarios, `PreparedStatement` descarga mucho trabajo a la base de datos, haciendo que todo vaya más fluido.



3.3. Ejecución de consultas (DQL)

En el mundo de SQL, cuando solo queremos leer datos sin modificar nada, hablamos de **DQL (Data Query Language)**. La sentencia reina aquí es el `SELECT`.

En Java, para ejecutar este tipo de sentencias, existe un método específico que **siempre** debes usar: **`executeQuery()`**.

1. El método `executeQuery()`

Este método tiene una característica fundamental: **siempre devuelve un objeto**. Concretamente, devuelve un `ResultSet` que contiene las filas encontradas en la base de datos (o ninguna, si la búsqueda no tuvo éxito).

- **Entrada:** Una sentencia SQL `SELECT`.
- **Salida:** Un objeto `ResultSet` (una tabla virtual en memoria).
- **Prohibido:** No uses este método para INSERT, UPDATE o DELETE. Si lo haces, Java lanzará una excepción indicando que la sentencia no produce un conjunto de resultados.

2. Sintaxis con `PreparedStatement`

Recuerda que, al usar `PreparedStatement` (nuestra opción segura), la consulta SQL ya se entregó al crear el objeto. Por tanto, al llamar a `executeQuery()`, **no debes pasarle ningún String entre los paréntesis**.

Error Común de Principiante: `pstmt.executeQuery(sql);` <-- ¡MAL! En `PreparedStatement` el método va vacío: `pstmt.executeQuery();`

3. Ejemplo Práctico: Buscar un Usuario

Vamos a buscar los datos de un usuario basándonos en su email. Fíjate cómo combinamos todo lo aprendido: `try-with-resources`, `PreparedStatement` y el método `executeQuery`.

Java
```
String sql = "SELECT id, nombre, fecha_registro FROM usuarios WHERE
email = ?";

try (
    Connection conn = DriverManager.getConnection(url, user, pass);
    PreparedStatement pstmt = conn.prepareStatement(sql);
) {
    // 1. Rellenamos el hueco (?) con el dato real
    pstmt.setString(1, "cliente@ejemplo.com");

    // 2. Ejecutamos la CONSULTA (DQL)
    // Fíjate: devuelve un ResultSet, pero el método NO lleva
parámetros aquí
    try (ResultSet rs = pstmt.executeQuery()) {

        // Aquí dentro es donde procesaremos los datos (lo veremos
en el punto 9.4)
        if (rs.next()) {
            System.out.println("Usuario encontrado.");
        } else {
            System.out.println("No existe usuario con ese email.");
        }
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

3.4. Ejecución de comandos de actualización (DML)

Si `SELECT` es "preguntar", entonces `INSERT`, `UPDATE` y `DELETE` son "actuar". Estas sentencias pertenecen al grupo **DML (Data Manipulation Language)** porque alteran el estado de los datos.

Para todas estas operaciones (y también para crear tablas con `CREATE`), usamos un único método universal: **`executeUpdate()`**.

1. El método `executeUpdate()`

A diferencia de las consultas de lectura, aquí la base de datos no tiene que devolvernos filas ni columnas. Lo único que necesitamos saber es **cuántas filas han sufrido cambios**.

- **Entrada:** Una sentencia SQL de acción (INSERT, UPDATE, DELETE).
- **Salida:** Un número entero (`int`).
    - Representa el número de **filas afectadas**.
    - Si devuelve `1`, se modificó una fila.
    - Si devuelve `50`, se modificaron cincuenta.
    - Si devuelve `0`, la sentencia se ejecutó correctamente (sin error), pero **no encontró nada** que modificar (por ejemplo, intentaste borrar un ID que no existía).

2. Ejemplos Prácticos

Veamos cómo se aplica este método a las tres operaciones clásicas. Recuerda: usamos `PreparedStatement` para inyectar los valores de forma segura.

A. Insertar datos (INSERT)

Queremos registrar un nuevo producto en el almacén.

Java
```
String sql = "INSERT INTO productos (nombre, precio, stock) VALUES
(?, ?, ?)";

try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

    // Rellenamos los datos
    pstmt.setString(1, "Monitor 24 pulgadas");
    pstmt.setDouble(2, 199.99);
    pstmt.setInt(3, 50);

    // Ejecutamos
    int filasAfectadas = pstmt.executeUpdate();

    if (filasAfectadas > 0) {
        System.out.println("¡Éxito! Producto registrado.");
    }

} catch (SQLException e) {
    System.err.println("Error al insertar: " + e.getMessage());
}
```

B. Modificar datos (UPDATE)

Queremos subir el precio de un producto específico (por su ID).

Java
```
String sql = "UPDATE productos SET precio = ? WHERE id = ?";

try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

    pstmt.setDouble(1, 250.00); // Nuevo precio
    pstmt.setInt(2, 5);         // ID del producto a cambiar

    int filas = pstmt.executeUpdate();

    if (filas > 0) {
        System.out.println("Precio actualizado correctamente.");
    } else {
        // Esto pasa si el ID 5 no existe en la base de datos
        System.out.println("No se encontró ningún producto con ID
5.");
    }
}
```

C. Borrar datos (DELETE)

Queremos eliminar a un usuario despedido.

Java
```
String sql = "DELETE FROM empleados WHERE dni = ?";

try (PreparedStatement pstmt = conn.prepareStatement(sql)) {

    pstmt.setString(1, "12345678X");

    int filas = pstmt.executeUpdate();

    if (filas > 0) {
        System.out.println("Empleado eliminado.");
    } else {
        System.out.println("No se pudo borrar: DNI no
encontrado.");
    }
}
```

Resumen de Métodos de Ejecución

Es vital no confundir los dos métodos principales. Aquí tienes la tabla definitiva para saber cuál usar:

Mi SQL empieza por...	Método a usar	¿Qué devuelve?
SELECT	`executeQuery()`	**`ResultSet`** (Los datos encontrados)
INSERT	`executeUpdate()`	**`int`** (Nº filas creadas)

Mi SQL empieza por...	Método a usar	¿Qué devuelve?
UPDATE	`executeUpdate()`	`int` (Nº filas cambiadas)
DELETE	`executeUpdate()`	`int` (Nº filas borradas)
CREATE / DROP	`executeUpdate()`	`int` (Suele ser 0)

4. Recuperación y Procesamiento de Información

4.1. El objeto `ResultSet`: Concepto de cursor y navegación

El `ResultSet` (Conjunto de Resultados) representa la tabla de datos que ha generado tu consulta. Sin embargo, tiene una particularidad vital: **actúa como una ventana deslizante**.

Imagina que has pedido el listado de todos los clientes y son 1 millón. Java no carga el millón de clientes en la memoria RAM de golpe (tu ordenador explotaría). En su lugar, el `ResultSet` mantiene un puntero abierto hacia la base de datos y va trayendo los datos a medida que tú los pides.

1. El Cursor (El Puntero)

El componente clave del `ResultSet` es el **cursor**. Es un apuntador interno que indica qué fila estamos leyendo en este momento.

La regla de oro es entender la posición inicial:

Cuando recibes el objeto ResultSet, el cursor no está en la primera fila. Está situado "Before First" (Antes de la primera fila, en el encabezado).

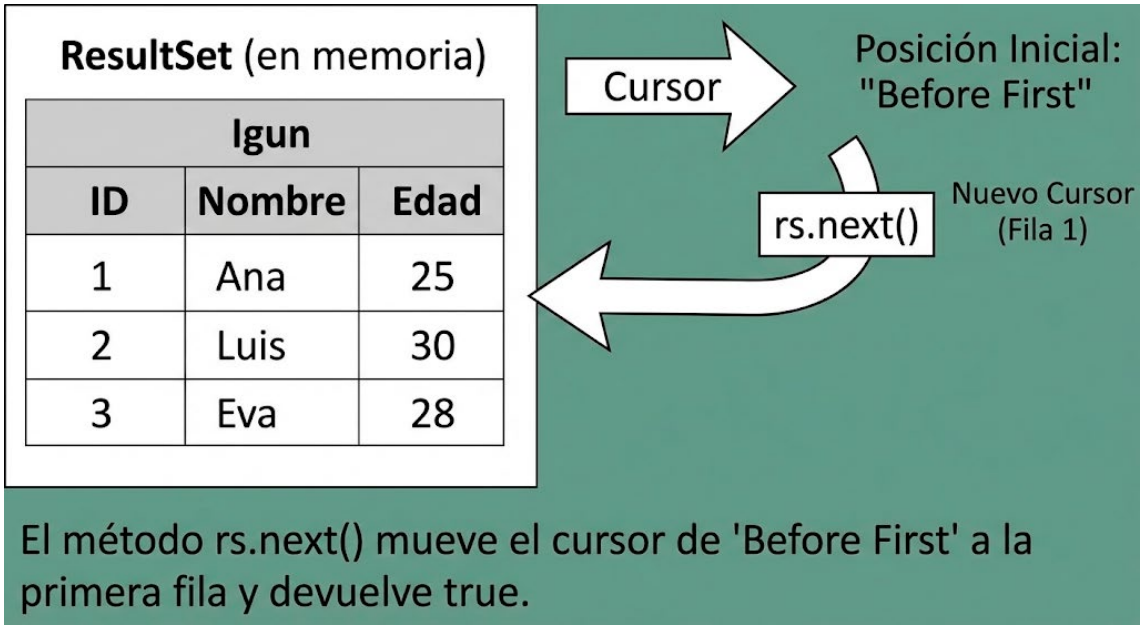
Si intentas leer datos nada más recibir el objeto, obtendrás un error, porque el cursor no apunta a ninguna fila válida todavía.

2. El método `next()` (El Motor)

Para mover el cursor, utilizamos el método **`rs.next()`**. Este método hace dos cosas simultáneamente:

1. **Avanza el cursor** una posición hacia adelante (a la siguiente fila).
2. **Devuelve un booleano (`true`/`false`)**:

   - **`true`**: "Me he movido y he encontrado una fila válida. Puedes leer datos".
   - **`false`**: "He llegado al final (`After Last`). No hay más datos".



3. Patrones de Navegación

Dependiendo de cuántos resultados esperes, usarás el `next()` con un `if` o con un `while`.

A. Para un solo resultado (Ej: Buscar por ID). Si buscas un usuario por su clave primaria, sabes que solo habrá (como máximo) un resultado.

Java
```
String sql = "SELECT * FROM usuarios WHERE id = 100";
// ... (código de conexión) ...
ResultSet rs = stmt.executeQuery(sql);

// Usamos IF porque solo esperamos una fila
if (rs.next()) {
    System.out.println("Usuario encontrado. Ahora puedo leer sus
datos.");
} else {
    System.out.println("El cursor llegó al final inmediatamente. El
usuario 100 no existe.");
}
```

B. Para múltiples resultados (Ej: Listar todos). Este es el patrón más común. Usamos un bucle while que sigue girando mientras next() devuelva true.

Java
```
String sql = "SELECT * FROM productos";
// ...
ResultSet rs = stmt.executeQuery(sql);

// El bucle estándar de JDBC
while (rs.next()) {
    // En cada vuelta del bucle, el cursor está en una fila
distinta.
    // Vuelta 1: Fila 1
```

```
    // Vuelta 2: Fila 2
    // ...
    System.out.println("Procesando fila...");
}
// Cuando sale del bucle, es que ya no hay más productos.
```

Tipos de ResultSet (Avanzado)

Por defecto, el cursor de JDBC es:

- **Forward Only (Solo avance):** Solo puedes ir hacia abajo (del 1 al 10). No puedes volver atrás (del 10 al 9).
- **Read Only (Solo lectura):** No puedes modificar los datos directamente en el objeto `ResultSet`.

Mas info: Existen ResultSets "scrollables" (que permiten `previous()`, `first()`, `last()`), pero consumen muchos más recursos y rara vez se usan en aplicaciones web modernas. Lo estándar es leer de arriba a abajo una sola vez.

4.2. Obtención de datos: Métodos `getXXX()` y Mapeo de Tipos

El objeto `ResultSet` no guarda la información como objetos genéricos. Sabe distinguir si una columna es un número, un texto o una fecha. Por eso, para sacar datos, no existe un método `get()` a secas.

Debes usar una familia de métodos **"Getter Tipados"**. Tú eres el responsable de saber qué tipo de dato hay en la base de datos y pedir el equivalente correcto en Java.

1. Dos formas de pedir los datos

Para indicar de qué columna quieres sacar el valor, tienes dos opciones:

A. Por Nombre de Columna (Recomendado)

Le pasas el nombre exacto de la columna en la tabla (o el alias si usaste `AS` en el SQL).
Java
```
String nombre = rs.getString("nombre_cliente");
```

- **Ventaja:** El código es muy legible. No importa si cambias el orden de las columnas en el `SELECT`.
- **Desventaja:** Ligeramente más lento (Java tiene que buscar el nombre), aunque inapreciable hoy en día.

B. Por Índice de Columna (Rápido pero peligroso)

Le pasas la posición numérica de la columna en la query. **¡OJO!** En JDBC, **los índices empiezan en 1**, no en 0.

Java

```
String nombre = rs.getString(2); // Dame la segunda columna del
SELECT
```

- **Ventaja:** Muy eficiente.
- **Desventaja:** Si cambias tu SQL de `SELECT id, nombre` a `SELECT nombre, id`, tu código se romperá o leerá datos erróneos.

2. Tabla de Mapeo: SQL vs. Java

¿Qué método debo usar? Depende del tipo de dato definido en la base de datos (DDL). Aquí tienes la traducción estándar:

Tabla de Mapeo: Tipos SQL vs. Java y Métodos getXXX()

Tipo en Base de Datos (SQL)		Tipo en Java		Método ResultSet a usar
CHAR, VARCHAR, TEXT	→	String	→	getString("col")
INT, INTEGER, SMALLINT	→	int	→	getInt("col")
BIGINT	→	long	→	getLong("col")
FLOAT, DOUBLE, DECIMAL	→	double	→	getDouble("col")
BIT, BOOLEAN, TINYINT(1)	→	boolean	→	getBoolean("col")
DATE	→	java.sql.Date	→	getDate("col")
TIMESTAMP, DATETIME	→	java.sql.Timestamp	→	getTimestamp("col")

Más info: Aunque puedes usar `getString()` para obtener un número (y Java lo convertirá a texto "123"), lo correcto es usar el tipo nativo para poder operar matemáticamente con él.

3. Ejemplo Completo de Extracción

Imagina una tabla `empleados` con las columnas: `id` (INT), `nombre` (VARCHAR), `salario` (DOUBLE) y `es_jefe` (BOOLEAN).

Java
```
String sql = "SELECT id, nombre, salario, es_jefe FROM empleados";
ResultSet rs = stmt.executeQuery(sql);

while (rs.next()) {
    // Extraemos cada columna usando el método adecuado
    int id = rs.getInt("id");
    String nombre = rs.getString("nombre");
    double salario = rs.getDouble("salario");
    boolean esJefe = rs.getBoolean("es_jefe"); // Traduce 1/0 a
true/false automáticamente

    // Ya tenemos variables Java normales para trabajar
    System.out.println("Empleado: " + nombre + " gana " + salario +
"€");
```

```
}
```

4. El peligro de los NULOS (`NULL`)

Este es un error técnico muy frecuente.

En SQL, una columna numérica (`INT`) puede valer `NULL` (vacío). Pero en Java, una variable primitiva `int` **no puede ser null**.

¿Qué pasa si haces `rs.getInt("edad")` y en la base de datos la edad es `NULL`?

- JDBC **NO** lanza una excepción.
- JDBC devuelve `0` (el valor por defecto del primitivo).

Esto es peligroso: no sabrás si el usuario tiene 0 años o si no introdujo su edad.

Solución: El método `wasNull()`. Si tu columna admite nulos, debes comprobarlo inmediatamente después de leer el dato.

Java
```
int edad = rs.getInt("edad"); // Si es NULL, edad valdrá 0

if (rs.wasNull()) {
    System.out.println("Edad: No especificada");
} else {
    System.out.println("Edad: " + edad);
}
```

4.3. Metadatos: Obtención de información sobre la estructura

La palabra "Metadato" significa "Datos sobre los datos".

- **Dato:** "Juan", "25", "Madrid".
- **Metadato:** "Esa columna se llama 'Nombre'", "Es de tipo Texto", "Tiene un ancho máximo de 50 caracteres".

En JDBC, el objeto encargado de esta información es **`ResultSetMetaData`**.

1. ¿Cómo se obtiene?

No se crea con `new`. Se le pide al propio `ResultSet` una vez que la consulta se ha ejecutado.

Java
```
ResultSet rs = stmt.executeQuery("SELECT * FROM clientes");

// Obtenemos el objeto que describe la estructura de lo que
acabamos de leer
ResultSetMetaData meta = rs.getMetaData();
```

2. Métodos Clave

Una vez tienes el objeto `meta`, puedes preguntarle casi todo sobre la forma de la tabla resultante. Los métodos más usados son:

Método	Descripción
`getColumnCount()`	Devuelve el **número total de columnas** recuperadas. Fundamental para hacer bucles `for`.
`getColumnName(int i)`	Devuelve el **nombre** de la columna (ej: "precio_venta").
`getColumnTypeName(int i)`	Devuelve el **tipo de dato en SQL** (ej: "VARCHAR", "INTEGER").
`getTableName(int i)`	Te dice a qué **tabla** pertenece esa columna (útil si hiciste un JOIN de varias tablas).

Recordatorio Importante: Al igual que en el `ResultSet`, en los Metadatos **los índices de columna empiezan en 1**, no en 0.

3. Ejemplo Práctico: El "Impresor Universal"

Vamos a crear un trozo de código capaz de imprimir por consola el resultado de **CUALQUIER** consulta SQL, sin importar si pedimos clientes, productos o facturas. El código se adapta solo.

Java
```
String sql = "SELECT * FROM productos"; // Podrías cambiar esto por
cualquier otra tabla

try (
    Connection conn = DriverManager.getConnection(url, user, pass);
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery(sql)
) {
    // 1. Obtenemos los metadatos
    ResultSetMetaData metaData = rs.getMetaData();

    // 2. Averiguamos cuántas columnas hay
    int numeroColumnas = metaData.getColumnCount();

    // 3. Imprimimos los ENCABEZADOS (Nombres de columna)
    System.out.println("--- RESULTADOS DE LA CONSULTA ---");
    for (int i = 1; i <= numeroColumnas; i++) {
        // Imprimimos el nombre + un tabulador (\t)
        System.out.print(metaData.getColumnName(i).toUpperCase() +
"\t");
    }
    System.out.println("\n-----------------------------------");

    // 4. Imprimimos los DATOS (Filas)
    while (rs.next()) {
        // Por cada fila, recorremos todas las columnas
dinámicamente
        for (int i = 1; i <= numeroColumnas; i++) {
```

```
            // Usamos getObject para que valga para String, Int,
Date, etc.
            System.out.print(rs.getObject(i) + "\t");
        }
        System.out.println(); // Salto de línea al terminar la fila
    }

} catch (SQLException e) {
    e.printStackTrace();
}
```

Resultado en consola:

Plaintext
```
--- RESULTADOS DE LA CONSULTA ---
ID      NOMBRE          PRECIO      STOCK
-----------------------------------
1       Monitor LED     150.0       20
2       Teclado USB     12.5        50
3       Ratón Óptico    8.99        100
```

Si cambias la SQL a `SELECT dni, apellidos FROM empleados`, el código anterior seguirá funcionando perfectamente y adaptará los encabezados. Esa es la magia de la programación genérica usando metadatos.

5. Manipulación Avanzada de Información

5.1. Transacciones en Java: `setAutoCommit`, `commit` y `rollback`

Una **Transacción** es un conjunto de varias operaciones SQL que deben tratarse como **una única unidad indivisible**.

La Regla de Oro (Atomicidad): O se ejecutan **TODAS** con éxito, o no se ejecuta **NINGUNA**. No puede haber "medias tintas".

El Ejemplo Clásico: La Transferencia Bancaria

Imagina que quieres transferir 500€ de la Cuenta A a la Cuenta B. Esto implica dos operaciones SQL:

1. `UPDATE cuentas SET saldo = saldo - 500 WHERE id = 'A'` (Restar dinero)
2. `UPDATE cuentas SET saldo = saldo + 500 WHERE id = 'B'` (Sumar dinero)

El Desastre sin Transacciones: Si ejecutas la línea 1 y, justo después, se va la luz o falla la red antes de ejecutar la línea 2, el dinero ha desaparecido de A pero nunca llegó a B. El dinero se ha "evaporado". Un error imperdonable.

1. El Modo por Defecto: `AutoCommit = true`

Por defecto, JDBC funciona en modo "Auto-Guardado".

- Cada vez que llamas a `executeUpdate`, JDBC envía un "COMMIT" automático a la base de datos.
- El cambio se hace permanente instantáneamente. No hay vuelta atrás.

Para manejar transacciones, lo primero que debemos hacer es **desactivar este comportamiento**.

2. Los Tres Mosqueteros de la Transacción

Estos métodos pertenecen al objeto **`Connection`**, no al `Statement`.

A. `setAutoCommit(false)`

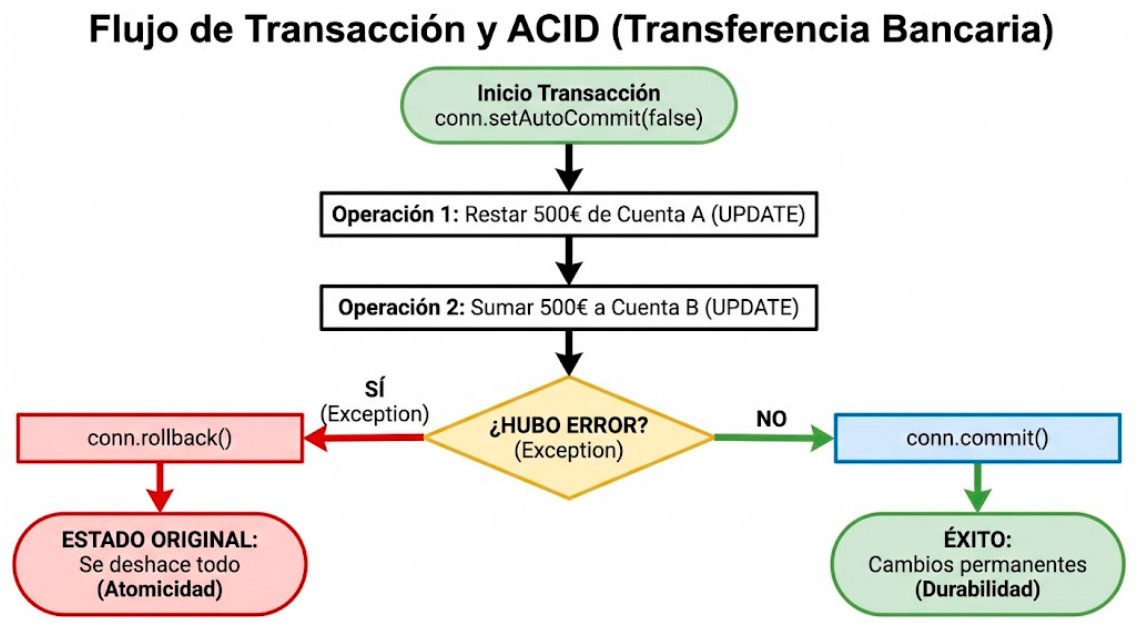
Es como quitar el "Guardado Automático" de un documento de Word. A partir de este momento, todo lo que hagas (`INSERT`, `UPDATE`, `DELETE`) ocurrirá en una zona temporal. La base de datos "toma nota" de los cambios, pero nadie más puede verlos y todavía no son definitivos en el disco duro.

B. `commit()` (Confirmar)

Es el botón de "Guardar". Le dice a la base de datos: "Todas las operaciones anteriores han salido bien. Hazlas permanentes ahora". En el ejemplo del banco, esto se ejecutaría solo si la resta y la suma funcionaron.

C. `rollback()` (Deshacer)

Es el botón de "Deshacer" (Ctrl+Z). Le dice a la base de datos: "Ha habido un error. Olvida todo lo que he hecho desde el último commit y deja la base de datos exactamente como estaba al principio".



3. El Patrón de Código (Template)

Para implementar esto correctamente, necesitamos una estructura `try-catch` muy específica.

Java
```
import java.sql.*;

public class TransaccionBancaria {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/banco";
        String user = "cajero";
        String pass = "seguro";

        Connection conn = null; // La declaramos fuera para usarla
en el catch

        try {
            conn = DriverManager.getConnection(url, user, pass);

            // 1. DESACTIVAMOS EL AUTO-GUARDADO (Inicio de
Transacción)
            conn.setAutoCommit(false);

            // --- OPERACIÓN 1: Retirar dinero de A ---
            PreparedStatement retirar = conn.prepareStatement(
                "UPDATE cuentas SET saldo = saldo - ? WHERE id =
?");
            retirar.setDouble(1, 500);
            retirar.setString(2, "Cuenta_A");
            retirar.executeUpdate();

            // Simulamos un error fatal (ej: se cae el sistema)
            // if (true) throw new RuntimeException("¡Error
inesperado en la red!");

            // --- OPERACIÓN 2: Ingresar dinero en B ---
            PreparedStatement ingresar = conn.prepareStatement(
                "UPDATE cuentas SET saldo = saldo + ? WHERE id =
?");
            ingresar.setDouble(1, 500);
            ingresar.setString(2, "Cuenta_B");
            ingresar.executeUpdate();

            // 2. SI LLEGAMOS AQUÍ, TODO FUE BIEN: CONFIRMAMOS
            conn.commit();
            System.out.println("Transferencia realizada con
éxito.");

        } catch (Exception e) {
            // 3. SI HUBO CUALQUIER ERROR: DESHACEMOS TODO
            try {
                if (conn != null) {
                    System.err.println("Error detectado.
Deshaciendo cambios (Rollback)...");
                    conn.rollback();
                }
            } catch (SQLException ex) {
```

```
                System.err.println("Error crítico al hacer
rollback: " + ex.getMessage());
            }
            e.printStackTrace();
        } finally {
            // 4. LIMPIEZA
            try {
                if (conn != null) {
                    // Buena práctica: Restaurar el modo por
defecto antes de cerrar/devolver la conexión
                    conn.setAutoCommit(true);
                    conn.close();
                }
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Análisis del Flujo

1. **Inicio:** `setAutoCommit(false)`. La base de datos entra en "modo espera".
2. **Operaciones:** Se restan 500€ de A. En este instante, si consultaras la base de datos desde otra consola, la cuenta A seguiría teniendo el dinero original. El cambio está "en el limbo".
3. **Bifurcación:**

    - **Éxito:** Se suma a B. Se ejecuta `commit()`. Los cambios se aplican y se vuelven visibles para todos.
    - **Fallo:** Salta una excepción en cualquier línea intermedia. El flujo va al `catch`. Se ejecuta `rollback()`. La resta de 500€ de A se cancela. El dinero nunca se perdió.

5.2. Integridad de los datos: Asegurar la consistencia

La **Integridad de Datos** significa que la información almacenada en la base de datos es **correcta, precisa y fiable**.

Cuando operamos sobre una sola tabla (ej: cambiar el nombre de un usuario), es difícil romper nada. Pero en el mundo real, una operación de negocio (como "Realizar una Compra") implica tocar **múltiples tablas** simultáneamente.

1. El escenario de riesgo: "La Compra Online"

Imagina que estás programando el backend de Amazon. Cuando un usuario pulsa "Comprar", ocurren tres cosas en tres tablas distintas:

1. **Tabla `PEDIDOS`:** Se crea un nuevo registro con el ID de compra.
2. **Tabla `DETALLES_PEDIDO`:** Se insertan los 5 productos que compró.
3. **Tabla `INVENTARIO`:** Se resta la cantidad comprada del stock disponible.

¿Qué pasa sin integridad? Si el paso 1 y 2 funcionan, pero el paso 3 falla (porque se cayó la red), tienes un problema grave:

- El pedido existe y se cobrará.
- El almacén **no sabe** que tiene que enviar los productos (el stock sigue igual).
- Resultado: Un cliente furioso y una base de datos "sucia" (inconsistente).

2. Las Propiedades ACID

Para evitar esto, las bases de datos relacionales (y tu código Java mediante JDBC) deben cumplir las reglas **ACID**. Son los cuatro pilares de la integridad en operaciones complejas.

A - Atomicidad (Atomicity)

- **Concepto:** "Todo o nada".
- **En Java:** Es lo que logramos con el bloque `conn.rollback()`.
- **Ejemplo:** O se crean el pedido, los detalles Y se baja el stock, o no se hace nada. No pueden quedar "pedidos a medias".

C - Consistencia (Consistency)

- **Concepto:** La base de datos nunca debe romper sus propias reglas (restricciones).
- **En Java:** Si intentas insertar un pedido para un `cliente_id = 999` y ese cliente no existe, la base de datos (gracias a las Foreign Keys) bloqueará la transacción.
- **Tu responsabilidad:** Java debe capturar esa `SQLException` y avisar al usuario de que la operación es inválida.

I - Aislamiento (Isolation)

- **Concepto:** "Lo que pasa en Las Vegas, se queda en Las Vegas".
- **El Problema de la Concurrencia:** Imagina que quedan **1** unidad de un producto.
    - Usuario A intenta comprarlo (inicia transacción).
    - Usuario B intenta comprarlo (inicia transacción milisegundos después).
- **La Solución:** Mientras la transacción de A no haga `commit()`, el Usuario B no debe ver el cambio de stock ni poder "robarle" el producto. La base de datos "bloquea" (locks) las filas afectadas temporalmente.

D - Durabilidad (Durability)

- **Concepto:** "Lo escrito, escrito está".
- **En Java:** Una vez que tu código ejecuta `conn.commit()` y no da error, tienes la garantía absoluta de que los datos están grabados en el disco duro. Aunque se vaya la luz un segundo después, al reiniciar el servidor, la compra estará ahí.

Resumen

1. Los Pilares de JDBC (Arquitectura)

- **JDBC (Java Database Connectivity):** Es un traductor universal. Permite que tu código Java hable con cualquier base de datos (MySQL, Oracle, Postgres) usando el mismo código estándar.
- **El Driver:** Es la pieza clave (un archivo `.jar`). Es el software específico del fabricante que realiza la traducción real. Sin driver, no hay conexión.
- **Interfaces Clave:**
  - `Connection`: La sesión (el túnel).
  - `Statement`: El vehículo que lleva la SQL.
  - `ResultSet`: La caja que trae los datos de vuelta.

2. Estableciendo la Conexión

- Usamos **`DriverManager.getConnection(url, user, pass)`**.
- **URL JDBC:** La dirección con formato específico (`jdbc:mysql://localhost:3306/mi_db`).
- **Manejo de Recursos:** Es obligatorio cerrar las conexiones. La forma moderna y segura es usar **`try-with-resources`** para evitar fugas de memoria y bloqueos en el servidor.

3. Ejecución de Sentencias (La Regla de Oro)

Aprendimos la diferencia crítica entre dos formas de enviar órdenes:

- **`Statement`:** Concatenación de Strings (`"id=" + id`). **Inseguro**, lento y difícil de leer.
- **`PreparedStatement`:** Uso de plantillas con interrogantes (`"id = ?"`).
  - **Seguridad:** Evita ataques de **Inyección SQL**.
  - **Rendimiento:** La base de datos precompila y cachea el plan de ejecución.
  - **Conclusión:** Debes usar `PreparedStatement` siempre que haya variables.

4. Métodos de Ejecución

Dependiendo de la SQL, usamos un método u otro:

Acción SQL	Método Java	Retorno
SELECT (Leer)	`executeQuery()`	Devuelve un **`ResultSet`**.
INSERT, UPDATE, DELETE	`executeUpdate()`	Devuelve un **`int`** (filas afectadas).

5. Recuperación de Datos (`ResultSet`)

- El `ResultSet` actúa como una ventana deslizante con un **cursor**.
- Empezamos "antes del primero". Usamos **`while(rs.next())`** para avanzar fila a fila.

- Extraemos los datos con métodos tipados: `getInt("id")`, `getString("nombre")`, `getDate("fecha")`.
- Usamos **Metadatos** (`ResultSetMetaData`) si necesitamos saber dinámicamente cuántas columnas hay o cómo se llaman.

6. Integridad y Transacciones

Para operaciones de negocio complejas (como transferencias bancarias), no basta con ejecutar comandos sueltos.

- **ACID:** Propiedades que garantizan la integridad (Atomicidad, Consistencia, Aislamiento, Durabilidad).
- **Control Manual:**

  1. `conn.setAutoCommit(false);` (Empezar transacción).
  2. `conn.commit();` (Guardar todo si hubo éxito).
  3. `conn.rollback();` (Deshacer todo si hubo error).

Chuleta Rápida de Código (Snippet)

Java

```
String sql = "SELECT nombre FROM usuarios WHERE edad > ?";
String url = "jdbc:mysql://localhost:3306/empresa";

// 1. TRY-WITH-RESOURCES (Abre y cierra solo)
try (Connection conn = DriverManager.getConnection(url, "user",
"pass");
     PreparedStatement pstmt = conn.prepareStatement(sql)) {

    // 2. RELLENAR HUECOS (PreparedStatement)
    pstmt.setInt(1, 18);

    // 3. EJECUTAR Y LEER (ResultSet)
    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            System.out.println("Usuario: " +
rs.getString("nombre"));
        }
    }
} catch (SQLException e) {
    e.printStackTrace(); // Gestión de errores
}
```